

Letný report

V letnom semestri som najprv vytvoril bruteforce riešenie pre zadanú úlohu pomocou AllSAT, pričom som hľadal perfektné párenie s minimálnou Hammingovou vzdialenosťou od pôvodného farbenia. Toto riešenie bolo neskôr optimalizované pomocou lineárneho programovania, čo algoritmus výrazne urýchlilo. Boli pridané rôzne pomocné nástroje, ako napríklad logovanie a export do formátu CSV. Bol vytvorený Python runner na automatické spúšťanie testovacích prípadov pre rôzne grafy.

Definície:

- **Konfigurácia** – dvojica vrcholov u, v taká, že $(u, v) \in E(G)$, teda hrana medzi nimi existuje v grafe G .
- **Farebná konfigurácia** – konfigurácia u, v spolu s farbením C .

Celý pipeline funguje nasledovne:

1. Python runner načíta graf vo formáte graph6 zo súboru.
2. Runner spustí binárny súbor s grafom:
 - 2.1. Nájde dvojicu u, v takú, že $(u, v) \in E(G)$.
 - 2.2. Vytvorí $G' = G - \{u, v\}$ - teda konfiguráciu.
 - 2.3. Nájde farbenie grafu G' pomocou SAT (CaDiCaL) - základné farbenie.
 - 2.4. Nájde riešenie pomocou ILP (HiGHS): perfektné párenie s defektom 3, s minimálnou Hammingovou vzdialenosťou od pôvodného farbenia.

Funkcia na meranie vzdialenosti perfektného párenia od pôvodného farbenia:

```
int hammingDistanceForDefect(Solution original,
                             Solution defect)
{
    int distance = 0;

    set<Edge> originalEdges = set<Edge>();

    for (const Edge &e: original.M1) originalEdges.
        insert(e);
    for (const Edge &e: original.M2) originalEdges.
        insert(e);
    for (const Edge &e: original.M3) originalEdges.
        insert(e);

    for (const Edge &e: originalEdges) {
        if (original.M1.contains(e) != defect.M1.
            contains(e)) {
            distance++;
        }

        if (original.M2.contains(e) != defect.M2.
            contains(e)) {
            distance++;
        }

        if (original.M3.contains(e) != defect.M3.
            contains(e)) {
            distance++;
        }
    }

    return distance;
}
```

Čo bolo dosiahnuté:

- Python runner na spúšťanie výpočtov grafov.
- Nájdenie odpovedí pre všetky kritické snarky s počtom vrcholov $n \leq 32$ (začalo sa na $n = 34$), jedno farbenie/konfigurácia. Výsledky boli exportované do `export_data/bulk`.
- Nájdenie odpovedí pre všetky kritické snarky s počtom vrcholov $n \leq 28$ (začalo sa na $n = 30$), všetky farbenia/konfigurácie.
- Testovanie: existuje brute-force riešenie, ktoré vypočíta všetky možné 3 perfektné párenia s defektom 3 a overí minimálnu vzdialenosť.
 - Pre kritický snark s počtom vrcholov $n = 10$ boli vykonané nasledujúce testy: jednoduché 1 farbenie a 1 konfigurácia, všetky farbenia / všetky konfigurácie.
- Bolo overených niekoľko väčších grafov, avšak pre ne sa brute-force prístup rýchlo stáva príliš pomalým na praktické použitie. Pre prípad $n = 10$ bola každá možná konfigurácia porovnaná s ILP riešením.